



UNIVERSIDAD DE CHILE
FACULTAD DE CIENCIAS FÍSICAS Y MATEMÁTICAS
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

EXTENSIÓN EXPERIMENTAL DE APPLICATIVO EN GHC MEDIANTE
REORDENAMIENTO DE MÓNADAS CONMUTATIVAS EN PROGRAMAS
PROBABILÍSTICOS DE HASKELL

MEMORIA PARA OPTAR AL TÍTULO DE
INGENIERO CIVIL EN COMPUTACIÓN

ESTEBAN ROMERO BERRÍOS

PROFESOR GUÍA:
FEDERICO OLMEDO B.

PROFESOR CO-GUÍA:
ISMAEL FIGUEROA P.

MIEMBROS DE LA COMISIÓN:
GONZALO NAVARRO B.
IGNACIO SLATER M.

SANTIAGO DE CHILE

2026

Resumen

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Una dedicatoria corta.

Agradecimientos

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Tabla de Contenido

1. Introducción	1
1.1. Contexto	1
1.2. Motivación	3
1.3. Objetivos	4
1.4. Contribuciones	5
2. Marco Teórico	6
2.1. Abstracciones Funcionales de Haskell	6
2.1.1. Functores	7
2.1.2. Aplicativos	8
2.1.3. Mónadas	10
2.2. Do-Notation	11
2.3. Mónadas Conmutativas	12
2.4. Mónadas de Probabilidad	12
2.5. ApplicativeDo	13
3. Problema	16
3.1. Limitación del Orden Sintáctico	16
3.2. Ejecución Secuencial	17
3.3. ApplicativeDo Actual	17
3.4. Plan con Reordenamiento	18
3.5. Formulación	18

4. Solución	20
4.1. Visión General	20
4.2. QualifiedDo	21
4.3. Información del Renamer	21
4.4. Dependencias Def-Use y Grafo RAW	22
4.5. Permutaciones	23
4.6. Evaluación de Candidatos	24
4.7. Selección	24
4.8. Integración en GHC	25
4.9. Selección Forzada	25
5. Validación	27
5.1. Metodología	27
5.2. Artefactos de Validación	27
5.3. Métricas	28
5.4. Corpus Sintético Maybe	29
5.5. Corpus Sintético Probabilístico	29
5.6. Corpus Real	30
5.7. Control IO	31
5.8. Resultados	31
5.9. Amenazas a la Validez	32
6. Conclusión	33
6.1. Resumen del Trabajo	33
6.2. Cumplimiento de Objetivos	33
6.3. Limitaciones	34
6.4. Trabajo Futuro	35
6.5. Cierre	35

Bibliografía	38
Apéndice A. Anexos Técnicos	39
A.1. Reproducibilidad	39
A.2. Pipeline de Validación	39
A.3. Logs y Trazas	39
A.4. Corpus	40
A.5. Detalles de Implementación	40

Índice de Tablas

Índice de Ilustraciones

2.1. Etapa de <i>rearrangement</i> propuesta por Marlow et al.	14
2.2. Etapa de <i>desugaring</i> propuesta por Marlow et al.	15

Capítulo 1

Introducción

1.1. Contexto

La programación probabilística es un paradigma que permite describir modelos con incertidumbre mediante programas que incorporan elecciones probabilísticas. A diferencia de los programas deterministas tradicionales, un programa probabilístico no representa únicamente un valor de salida, sino una distribución de posibles resultados. Esta característica permite razonar directamente sobre distribuciones de probabilidad, logrando así predecir fenómenos inciertos y realizar inferencia probabilística sobre eventos del mundo real.

Estos programas han demostrado ser fundamentales para diversas áreas de la computación, así como en ciberseguridad, mediante sus algoritmos de generación de números aleatorios, al igual que la criptografía, mediante sus protocolos de seguridad. Asimismo, en campos más recientes, este paradigma resulta clave para el entrenamiento de distintos modelos de Machine Learning y el desarrollo de sistemas de Inteligencia Artificial.

Cabe destacar que, cuando un programa con elecciones probabilísticas se ejecuta en un lenguaje de programación tradicional, el resultado observado suele corresponder a una sola muestra de la distribución inducida por el programa. Por ejemplo, si se modela el lanzamiento de una moneda, una ejecución concreta retornará únicamente uno de sus posibles resultados, como “cara” o “sello”. Sin embargo, desde una perspectiva semántica, el programa no describe solo ese resultado particular, sino una distribución sobre todos los resultados posibles.

En lenguajes funcionales puros como **Haskell**, es posible representar este tipo de cálculos mediante mónadas de probabilidad. En particular, para distribuciones discretas y finitas, estas mónadas permiten construir explícitamente la distribución inducida por el programa y razonar sobre todos sus posibles resultados. Este proceso corresponde a una forma de inferencia probabilística.

Gracias a la forma en la cual se implementan las mónadas de probabilidad de **Haskell**, es posible realizar consultas y operaciones sobre la distribución de salida de un programa. Por ejemplo, si se desea inferir la distribución asociada al lanzamiento de una moneda seguido del lanzamiento de un dado, la *do-notation* de las mónadas de **Haskell** permite usar la siguiente

sintaxis:

```
flipCoinRollDice = do
  c <- uniform [Head, Tail]
  d <- uniform [1 .. 6]
  return (c, d)
```

En el código anterior, se logra apreciar que la variable `c` codifica cada evento de la primera distribución de probabilidad (moneda) y, por otro lado, la variable `d` hace lo propio de manera análoga con la segunda (dado); al formar la tupla con ambos eventos, se obtiene la distribución correspondiente a la ejecución conjunta de los dos experimentos. Siendo más precisos, esta nueva distribución estaría compuesta por doce tuplas de eventos.

Tal como se aprecia en el ejemplo, la *do-notation* es especialmente conveniente para este tipo de programas: cada statement puede introducir una variable aleatoria o una distribución intermedia. Además, los statements posteriores pueden depender de los valores ya definidos anteriormente, como se observa en el segundo ejemplo:

```
do
  d1 <- uniform [1 .. 6]
  d2 <- uniform [1 .. 6]
  d3 <- uniform [1 .. d1]
  d4 <- uniform [1 .. d2]
  return (d3, d4)
```

En el ejemplo mostrado se modela la inferencia del lanzamiento de dos dados de forma simultánea `d3` y `d4`, con la particularidad de que la cantidad de caras de cada dado está determinada por el resultado obtenido en los primeros lanzamientos de `d1` y `d2`. Con esto, en la instrucción `return` se generará una nueva distribución con todas las tuplas posibles al lanzar los dados `d3` y `d4` después del proceso de selección de caras definido por `d1` y `d2`. Por lo que las probabilidades resultantes difieren de las habituales.

En este caso, para obtener la tupla $(6,6)$ en la distribución de salida, es necesario que en ambos lanzamientos de `d1` y `d2` se obtenga el número 6. Luego, se determina que los dados `d3` y `d4` tendrán 6 caras cada uno y, finalmente, deberán arrojar el número 6 cada uno por separado. En consecuencia, la tupla $(6,6)$ tiene una probabilidad asociada de $1/1296$.

El problema yace en que, el uso de la *do-notation* para representar programas probabilísticos no expone por sí sola todo el paralelismo disponible a lo largo del programa. En **Haskell**, un bloque `do` es azúcar sintáctica de una composición mediante el operador *bind* (`>>=`). Este operador permite que el cómputo posterior dependa del valor producido por el cómputo anterior, lo que resulta fundamental para modelar efectos y contextos como estado, entrada/salida, fallas, no determinismo o distribuciones probabilísticas [19]. Sin embargo, esa misma generalidad obliga a interpretar el bloque como una secuencia ordenada de efectos, haciendo que este sea inherentemente secuencial.

Una alternativa para resolver esto sería escribir el programa con otra sintaxis, pero manteniendo la misma semántica, de modo que la ejecución pueda ser paralelizada. Sin embargo, esto conllevaría la pérdida de la *do-notation*, la cual es ampliamente utilizada en **Haskell**; lo ideal sería que esta pudiera seguir utilizándose.

Esta tensión entre expresividad y paralelización ya ha sido abordada dentro del compilador estándar de **Haskell**, **GHC**, mediante la extensión **ApplicativeDo**, propuesta por Marlow et al. [13]. Esta extensión permite que el compilador analice un bloque escrito con *do-notation* y, cuando las dependencias entre statements lo permiten, lo transforme automáticamente a una expresión que utiliza operadores aplicativos como (**<\$>**) y (**<*>**). De este modo, el programador conserva la sintaxis monádica usual, mientras que el compilador puede generar un plan de ejecución con mayor estructura paralelizable.

La idea central de **ApplicativeDo** es distinguir qué partes de un bloque **do** requieren realmente encadenamiento monádico y cuáles pueden evaluarse de manera aplicativa. A diferencia del operador *bind*, una expresión aplicativa fija de antemano la estructura del cómputo, por lo que no necesita esperar el resultado de un statement para decidir la forma del siguiente. Esto permite que **GHC** aproveche independencia entre subcómputos sin exigir al programador abandonar la *do-notation*. Sin embargo, el algoritmo estándar de **ApplicativeDo** está diseñado para trabajar sobre mónadas arbitrarias, por lo que es conservador con la estructura original del programa, manteniendo el orden relativo de los statements escritos originalmente por el programador [7].

Ese orden es necesario para mónadas arbitrarias, porque los efectos pueden ser observables y el intercambio de statements podría cambiar el resultado final del programa. No obstante, en clases particulares de mónadas, como las mónadas conmutativas, ciertos subcómputos independientes entre sí pueden intercambiarse de lugar, alterando solo el orden sintáctico de la *do-notation* sin modificar la semántica original del programa. Esta memoria estudia precisamente esa oportunidad dentro de **GHC**: extender experimentalmente el mecanismo de **ApplicativeDo** para que, bajo una marca explícita de conmutatividad, el compilador pueda explorar reordenamientos semánticamente válidos de statements para luego evaluarlos mediante **ApplicativeDo**, seleccionando así el candidato de menor costo bajo el modelo adoptado.

1.2. Motivación

La motivación principal de esta memoria surge del costo asociado a la inferencia en programas probabilísticos. Cuando una mónada de probabilidad representa explícitamente una distribución discreta, cada combinación de elecciones probabilísticas puede aumentar la cantidad de resultados que deben considerarse. En este escenario, la posibilidad de exponer independencia entre subcómputos no es solo una mejora sintáctica, sino una oportunidad para obtener planes de ejecución más convenientes que usarían de mejor manera los recursos disponibles como el multihilo.

Aunque **ApplicativeDo** permite que **GHC** transforme fragmentos de *do-notation* a expresiones aplicativos, su análisis parte del orden escrito por el programador. Esto significa que

la independencia entre subcómputos puede existir semánticamente, pero permanecer oculta por la secuencia sintáctica original. En tales casos, el algoritmo estándar puede generar un plan de ejecución correcto, pero no necesariamente el mejor plan disponible si se permitieran reordenamientos seguros.

Una forma manual y directa de resolver este problema sería reescribir el programa para ubicar statements independientes en posiciones más favorables para `ApplicativeDo`. Sin embargo, esa solución traslada al programador una responsabilidad que podría abordarse en el compilador. Además, obliga a modificar la organización natural del programa, afectando la legibilidad y la mantenibilidad de la *do-notation*, precisamente una de las razones por las que esta sintaxis resulta atractiva en `Haskell`.

Por ello, resulta relevante estudiar si el criterio conservador de `ApplicativeDo` puede especializarse bajo una hipótesis semántica adicional. En particular, cuando la mónada utilizada es tratada explícitamente como conmutativa, ciertos statements independientes pueden intercambiarse sin alterar el significado del programa. Esta observación motiva extender experimentalmente el pipeline de `GHC` para explorar órdenes alternativos, evaluar cada uno mediante el algoritmo existente de `ApplicativeDo` y seleccionar aquel que ofrezca el menor costo bajo el modelo adoptado.

1.3. Objetivos

El objetivo general de esta memoria es extender experimentalmente la implementación de `ApplicativeDo` en `GHC`, de modo que pueda considerar reordenamientos semánticamente válidos de statements dentro de bloques `do` cuando la mónada usada es declarada como conmutativa. El propósito es ampliar el espacio de planes de ejecución evaluados por el compilador y seleccionar, bajo el modelo de costos adoptado, un plan no peor que el obtenido a partir del orden sintáctico original.

Los objetivos específicos son los siguientes:

1. Caracterizar el funcionamiento y las limitaciones de `ApplicativeDo` en `GHC` respecto del procesamiento de bloques `do` y la generación de planes de ejecución.
2. Diseñar un modelo de dependencias locales entre statements que permita identificar reordenamientos semánticamente válidos bajo una hipótesis de conmutatividad.
3. Implementar una extensión experimental que detecte bloques conmutativos, genere órdenes alternativos válidos y evalúe cada alternativa mediante el algoritmo existente de `ApplicativeDo`.
4. Integrar la extensión en una versión modificada de `GHC`, incorporando mecanismos explícitos de activación y control experimental.
5. Validar semánticamente la extensión sobre corpus sintéticos, comparando salidas observables y costos asociados a los planes de ejecución generados.

6. Construir y validar un corpus real de programas probabilísticos, definiendo criterios de selección, adaptación y comparación para evaluar la extensión fuera de casos sintéticos.

1.4. Contribuciones

Las principales contribuciones de esta memoria son:

- Una extensión experimental del Renamer de `GHC` que intercepta bloques `do` procesados por `ApplicativeDo` y, cuando corresponde, genera órdenes alternativos de statements.
- Un mecanismo de activación explícito basado en `QualifiedDo`: los programas importan el módulo agregado `Control.Monad.CommutativeDo` como `CD` y usan bloques `CD.do`; el módulo exporta un marcador explícito que indica al compilador que el bloque puede tratarse bajo la hipótesis de conmutatividad.
- La construcción de un grafo de precedencia con dependencias *Read-After-Write* a partir de la información disponible en el Renamer de `GHC`: variables definidas por cada statement y variables libres locales usadas por statements posteriores.
- La enumeración de ordenamientos topológicos válidos para ese grafo, junto con la evaluación de cada permutación mediante el algoritmo existente de `ApplicativeDo`.
- Un criterio de selección automática por costo mínimo y una flag experimental para forzar candidatos individuales durante la validación semántica.
- Corpus sintéticos de evaluación para `Maybe` y para `Dist.T Rational`, además de un pipeline reproducible que registra trazas, planes de ejecución, costos y resultados comparados.
- Un corpus real de programas probabilísticos seleccionado, adaptado y validado para evaluar la extensión fuera de casos sintéticos, considerando criterios de comparación semántica y costos asociados a los planes de ejecución generados.
- Documentación técnica, parches de `GHC` y scripts de validación disponibles en el repositorio de la memoria [1].

Capítulo 2

Marco Teórico

Este capítulo presenta los fundamentos conceptuales necesarios para comprender tanto el problema abordado en esta memoria como la solución desarrollada. En primer lugar, se estudian las principales abstracciones funcionales de **Haskell**: funtores, aplicativos y mónadas. Estas clases forman una jerarquía que permite estructurar cómputos dentro de un contexto y se distinguen por el grado de dependencia que pueden expresar entre las distintas partes de un programa.

A continuación, se profundiza en la *do-notation*, explicando su relación con la composición monádica y cómo sus statements de enlace se traducen, en su forma convencional, mediante el operador *bind* ($>>=$). Esto es fundamental para entender por qué **ApplicativeDo** puede reemplazar parte de una expresión monádica por una expresión aplicativa [13, 7].

Sobre esta base, se introduce la noción de mónada conmutativa, cuya propiedad relevante permite intercambiar ciertos cómputos independientes sin modificar la semántica del programa [10, 9]. Posteriormente, se presentan las mónadas de probabilidad como un caso canónico de esta propiedad y como el principal dominio de aplicación de la memoria.

Finalmente, se estudia la extensión **ApplicativeDo**, describiendo la transformación que realiza sobre bloques **do**, la forma en que identifica dependencias entre statements y la generación de expresiones aplicativas que pueden exponer una mayor estructura paralelizable. Este recorrido permite establecer los conceptos necesarios para analizar posteriormente las limitaciones del mecanismo actual y la extensión experimental propuesta.

2.1. Abstracciones Funcionales de Haskell

Una característica relevante de **Haskell** es que las funciones son valores de primera clase: pueden recibirse como argumentos, retornarse como resultados y almacenarse en estructuras de datos. Además, las funciones de varios argumentos se expresan de forma curried, es decir, como una sucesión de funciones que reciben un argumento a la vez. Esto permite aplicar una función parcialmente y obtener otra función como resultado.

Por ejemplo, el operador binario `(+)` posee el tipo `(+) :: Num a => a -> a -> a`, donde se establece que `a` puede ser cualquier tipo numérico. Al aplicarlo únicamente al valor `5` mediante la expresión `(+) 5`, se obtiene la función unaria `(5+)` de tipo `Num a => a -> a` producto de aplicar parcialmente `(+)`. En otras palabras, al aplicar este operador con un solo argumento, se retorna una nueva función que espera el argumento restante. Estas propiedades resultan fundamentales para las abstracciones estudiadas en esta sección, cuyas operaciones principales reciben, transforman o producen funciones.

Sobre esta base, `Functor`, `Applicative` y `Monad` forman una jerarquía de clases de tipos para estructurar cálculos dentro de un contexto. Estas abstracciones se distinguen por el grado de dependencia que permiten expresar entre las partes de un programa. Un functor permite transformar el resultado de un cálculo; un aplicativo permite combinar cálculos cuya estructura está determinada de antemano; y una mónada permite que un cálculo posterior dependa del valor producido por uno anterior [19, 15, 20].

2.1.1. Functores

En Haskell, un *functor* es una `typeclass` que representa a toda estructura que permita aplicar una función a los valores que contiene, sin la necesidad de cambiar la estructura en sí [20]; por esto, se puede imaginar un functor como un contenedor. A partir de esta noción, para ejecutar cálculos sobre los valores contenidos en él, se implementa la función canónica de los funtores, `fmap`. Esta función permite *desempaquetar* un functor, aplicar una función `f` sobre su contenido y luego *empaquetar* nuevamente el resultado. Además, `fmap` debe respetar las leyes de identidad y composición, lo cual garantiza que el mapeo preserve la estructura abstracta del contenedor [20].

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
```

Los ejemplos clásicos de funtores son las listas y árboles, debido a que, al aplicar `fmap` se transforman sus datos de manera uniforme, manteniendo la estructura subyacente.

Otro ejemplo relevante es `Maybe`. El tipo `Maybe a` representa valores opcionales: un valor disponible se expresa mediante el constructor de datos `Just`, mientras que la ausencia de un valor se representa mediante `Nothing`. Al utilizar `fmap` sobre este tipo, la función recibida se aplica al valor contenido en `Just`, mientras que `Nothing` permanece sin cambios.

En la implementación de `base` incluida con GHC, este comportamiento se define mediante la siguiente instancia de `Functor` para el constructor de tipos `Maybe`:

```
instance Functor Maybe where
  fmap _ Nothing = Nothing
  fmap f (Just a) = Just (f a)
```

La primera ecuación establece que, cuando el valor recibido es `Nothing`, no existe ningún elemento sobre el cual aplicar la función y, por tanto, el resultado permanece como `Nothing`.

El guion bajo indica que la función recibida no necesita utilizarse en este caso. En cambio, cuando el valor tiene la forma `Just a`, la segunda ecuación aplica la función `f` al elemento `a` y vuelve a encapsular el resultado mediante el constructor `Just`. Por ejemplo:

```
fmap (+1) (Just 2) = Just 3
fmap (+1) Nothing  = Nothing
```

En este caso, `Maybe a` corresponde a un tipo concreto, mientras que `Maybe` es el constructor de tipos para el cual se definen instancias de las clases `Functor`, `Applicative` y `Monad`. Por esta razón, se utilizará como ejemplo a lo largo de las subsecciones siguientes, permitiendo observar cómo un mismo contexto puede transformar valores, combinar cálculos y expresar dependencias entre sus resultados.

2.1.2. Aplicativos

La `typeclass` *Applicative* de `Haskell` caracteriza un estilo aplicativo de programación, más débil que las mónadas y, por tanto, aplicable a una mayor variedad de estructuras [15]. Esta abstracción extiende las capacidades de los funtores y permite aplicar funciones de varios argumentos a valores que ya se encuentran dentro de un contexto. Por ende, la necesidad de aplicar funciones más complejas a los funtores; como lo sería una función binaria, se ve resuelta. Para ilustrar este comportamiento, se utilizará nuevamente el tipo `Maybe` y el operador binario `(+)`.

Para resolver y aplicar la función antes mencionada, un functor aplicativo permite operar con valores que ya se encuentran dentro de un contenedor y aplicarlos a otro contenedor. Considerando el functor aplicativo concreto `Maybe Int`, en caso de sumar `Just 2` y `Just 3`, primero aplicamos `(+)` al primer contenedor, desempaquetando el número 2 y empaquetando la función ahora unaria `(2+)` en un nuevo aplicativo `Just (2+)` siendo de tipo `Maybe (Int -> Int)`. Luego, se aplica el contenido de este último al segundo contenedor, desempaquetando el número 3, realizando la operación y empaquetando el resultado 5 en un nuevo aplicativo `Just 5`.

Lo más destacable de los aplicativos es que permiten manejar cada abstracción específica dentro de sus propias implementaciones. Por ejemplo, para el tipo antes mencionado `Maybe Int`, si alguno de los dos sumandos es `Nothing`, se produce un cortocircuito que retorna al instante `Nothing`. De esta forma, `Maybe Int` actúa como un aplicativo que evita la necesidad de implementar sumas para cada aridad con múltiples líneas de *Pattern Matching* anidado para verificar si un sumando es `Just Int` o `Nothing`.

De este modo, los funtores aplicativos permiten aplicar funciones con un número arbitrario de argumentos sin dificultad. Estos mismos, se describen mediante la siguiente definición:

```
class Functor f => Applicative f where
  pure :: a -> f a
  <*> :: f (a-> b) -> f a -> f b
```

En particular, el *aplicativo* `Maybe a` considerando un tipo genérico `a`, implementa los operadores `pure` y `<*>` conforme a su abstracción funcional específica, es decir, trabajar con valores opcionales dentro de un programa, y combinar operaciones que pueden fallar, siendo su implementación concreta la siguiente:

```
instance Applicative Maybe where
  pure = Just
  Nothing <*> _ = Nothing
  _ <*> Nothing = Nothing
  (Just f) <*> (Just x) = Just (f x)
```

Nótese que, en caso de operar con algún fallo de cómputo o ausencia de algún valor, cuya abstracción equivale al constructor `Nothing`, las siguientes aplicaciones retornarán a su vez un elemento `Nothing`, modelando así el comportamiento de la clase `Maybe a`. En caso contrario, si ambos operandos se encuentran definidos, es decir, son elementos encapsulados en el constructor `Just`, se procede con la aplicación del primero sobre el segundo, para finalmente empaquetar el elemento resultante dentro del mismo constructor `Just`.

Volviendo al ejemplo de aplicación de la suma (+) sobre elementos de tipo `Maybe Int`, se requiere que esta se encuentre encapsulada dentro del constructor `Just`, ya que, si no lo estuviera, se rompería el invariante de desempaquetado y empaquetado asociado a la implementación del operador `<*>`. Para resolver esto, es preciso utilizar `pure`, que permite empaquetar la función `f` desde un principio. De este modo, la instancia concreta de la suma sobre `Maybe Int` tendría la siguiente sintaxis:

```
pure (+) <*> Just 2 <*> Just 3
```

Posteriormente, para que la notación sea más limpia y versátil, se introduce el operador `<$>`, el cual satisface la siguiente igualdad: `f <$> m = pure f <*> m`. Esto quiere decir que, dada una función no empaquetada `f` y un aplicativo `m`, se empaqueta la respectiva función con `pure` y luego se aplica. Por tanto, el ejemplo anterior se vería modificado, simplificando su sintaxis:

```
(+) <$> Just 2 <*> Just 3
```

Finalmente, nótese que en una expresión aplicativa, la estructura del cómputo queda fijada de antemano: cada argumento se evalúa en su propio contexto y luego se combinan sus resultados mediante la aplicación de una función pura. Esta característica hace que, a nivel semántico, los subcómputos asociados a los argumentos sean independientes entre sí (en el sentido de que ninguno puede *usar* el resultado del otro para definirse), aunque sus efectos todavía podrían conservar un orden observable. Esta estructura podría habilitar optimizaciones como evaluación paralela cuando la implementación lo permite [15].

Sin embargo, esta misma restricción implica que no es directo expresar dependencias entre argumentos. En particular, si se quisiera que el segundo argumento dependa del valor

producido por el primero, el estilo aplicativo resulta insuficiente, ya que la forma del cómputo no puede adaptarse dinámicamente a resultados intermedios. Asimismo, existen situaciones en que la función a aplicar es de tipo `a -> f b`; es decir, retorna un valor ya encapsulado en el contexto, lo que rompe la firma esperada requiriendo un mecanismo adicional, lo cual motiva la introducción de mónadas [19].

2.1.3. Mónadas

Las *mónadas* de Haskell son una `typeclass` que describe cómo encadenar operaciones que producen o manejan efectos, manteniendo un flujo de ejecución ordenado dentro de un contexto [19]. Esto nos ayuda a solucionar los dos problemas descritos anteriormente mediante el operador *bind*, el cual es escrito como `m a >>= f`. Este operador combina un cómputo `m a` con una función `a -> m b`, permitiendo construir el segundo cómputo a partir del resultado del primero. Para ciertas mónadas, se puede interpretar como el desempaquetar la mónada de la izquierda y aplicar la función de la derecha sobre el elemento `a`, de manera que lo retornado por la ejecución de `f a` sea nuevamente una mónada `m b`.

```
class Applicative m => Monad m where
  return :: a -> m a
  (>>=)  :: m a -> (a -> m b) -> m b
```

De esta manera, si queremos concatenar múltiples aplicaciones a distintas mónadas como con el operador `<*>`, hace falta declarar funciones anónimas λ que nos permitan representar un flujo claro del código. En vista de lo expuesto, resulta que el tipo `Maybe a` también es una mónada, y para ilustrar el comportamiento de estas mismas, se utilizará el mismo ejemplo de la suma (+) descrito en la sección anterior.

En este caso, la implementación de los operadores `return` y `>>=`, que satisfacen el comportamiento de la clase `Maybe a` con `a` un tipo genérico, es la siguiente:

```
instance Monad Maybe where
  return x = Just x
  Nothing >>= _ = Nothing
  (Just x) >>= f = f x
```

Por consiguiente, al igual que los aplicativos, la mónada `Maybe a` implementa el operador *bind* tal que respete la abstracción asociada a esta clase, es decir, trabajar con elementos opcionales. Por ende, en caso de aplicar una función a un elemento no definido, dicho de otro modo, que utiliza el constructor `Nothing`, se retornará este mismo valor. En contraste con lo anterior, si el elemento a desempaquetar es de tipo `Just`, se ejecuta la función `f` sobre este. En este caso, la función que se ejecuta debe respetar la firma de tipos y retornar una nueva mónada. Ya explicado esto, el código equivalente a la aplicación de la función (+) pero usando notación monádica es el siguiente:

```
Just 3 >>= (\x -> Just 2 >>= (\y -> return ((+) x y)))
```

Por tanto, al ejecutar el operador `>>=`, se desempaqueta el número 3 y se aplica la función anónima de la derecha sobre este valor, reemplazando todas las apariciones del identificador `x` por 3. A continuación, se realiza el mismo proceso con la mónada que contiene el 2, resultando finalmente en la instrucción `((+) 3 2)` que será empaquetada gracias a la instrucción `return` retornando la mónada `Just 5`.

Cabe recalcar que ahora el identificador `x` está disponible en todo el alcance de la primera función anónima. Por ende ahora el contexto del segundo argumento `Just 2` puede considerarlo para su propia definición. Un programa que use esta nueva expresividad de las mónadas se vería así:

```
Just 3 >>= (\x -> Just ((* 2 x) >>= (\y -> return ((+) x y)))
```

Del programa anterior, el primer cómputo desempaqueta `Just 3` y reemplaza este elemento en las apariciones del identificador `x`. Luego, el cómputo posterior `Just ((* 2 x)` es capaz de considerar el valor numérico de `x` retornando `Just 6` y a su vez desempaquear su resultado y reemplazar el identificador `y`. Quedando como cómputo final la operación `((+) 3 6)` cuyo resultado termina siendo empaquetado por la instrucción `return` como `Just 9`. Este flujo de ejecución permite una mayor expresividad enriqueciendo así el programa. De esta manera la notación monádica termina siendo algo característico y central en Haskell.

2.2. Do-Notation

La *do-notation* es una sintaxis de Haskell para escribir cálculos monádicos de forma secuencial. Un bloque `do` contiene statements que pueden vincular resultados intermedios y usarlos en los statements posteriores. Por ejemplo:

```
example :: Maybe Int
example = do
  x <- Just 1
  y <- Just (x + 1)
  return (x + y)
```

Intuitivamente, este bloque se traduce a una composición con `(>>=)`, donde el cuerpo posterior queda dentro de una función que recibe el valor producido por el statement anterior:

```
Just 1 >>= \x ->
Just (x + 1) >>= \y ->
return (x + y)
```

La consecuencia es que el orden de los statements no es solo una decisión estética. En el ejemplo, `y` depende de `x`, porque la expresión `Just (x + 1)` usa una variable definida antes. Si se intentara mover el segundo statement antes del primero, el programa dejaría de estar bien formado: `x` todavía no estaría en alcance.

Esta observación introduce la noción de dependencia *def-use*: un statement que usa una variable local debe mantenerse después del statement que la define. La extensión propuesta en esta memoria conserva esta restricción. Su aporte consiste en explorar cambios de orden solamente cuando no violan esas dependencias locales y cuando el bloque fue marcado bajo una hipótesis de conmutatividad.

2.3. Mónadas Conmutativas

Una mónada conmutativa es, en términos operacionales, una mónada donde ciertos cómputos independientes pueden intercambiarse sin cambiar el resultado observable. La noción aparece en el estudio categórico y semántico de efectos, y resulta especialmente natural en contextos probabilísticos donde dos elecciones independientes pueden combinarse en cualquier orden y describir la misma distribución conjunta [10, 9].

La propiedad relevante para esta memoria puede entenderse mediante el siguiente contraste. Si dos statements no comparten variables locales, existe independencia estructural: ninguno necesita el valor definido por el otro. Sin embargo, esa ausencia de dependencia de datos no garantiza por sí sola que sea seguro intercambiarlos. En una mónada como `IO`, dos acciones independientes desde el punto de vista de variables podrían imprimir mensajes en distinto orden, y ese cambio sería observable.

Por tanto, el grafo de dependencias usado por la extensión solo resuelve una parte del problema: identifica qué intercambios respetan el alcance de variables y las relaciones *def-use*. La validez semántica del reordenamiento completo requiere además una hipótesis sobre la mónada. En el diseño de esta memoria, esa hipótesis se declara explícitamente mediante un mecanismo de opt-in basado en `QualifiedDo`.

Es importante destacar que `GHC` no prueba que una mónada sea conmutativa. La clase y el marcador agregados por esta memoria actúan como una afirmación del programador: si un bloque se escribe con la notación conmutativa experimental, el compilador puede explorar reordenamientos que respeten dependencias RAW; si se usa `do` normal, el comportamiento conservador se mantiene.

2.4. Mónadas de Probabilidad

Una mónada de probabilidad permite interpretar un programa como una distribución de posibles resultados. En lugar de producir un único valor, el programa describe cómo se combinan elecciones probabilísticas y cómo se propagan sus probabilidades. Este enfoque ha sido usado para modelar distribuciones discretas en lenguajes funcionales y para estudiar

inferencia probabilística de manera composicional [3, 18].

Un ejemplo simple es el lanzamiento de una moneda seguido por el lanzamiento de un dado. En una notación monádica, cada statement puede representar una elección probabilística:

```
coinAndDie = do
  coin <- uniform [Head, Tail]
  die  <- uniform [1 .. 6]
  return (coin, die)
```

Si las elecciones son independientes, el resultado corresponde a la distribución conjunta sobre todos los pares posibles. En programas más complejos, una elección posterior puede depender del resultado de una elección anterior, lo que introduce dependencias de datos similares a las vistas en la *do-notation* monádica general.

En esta memoria, el corpus probabilístico usa la mónada `Dist.T Rational` del paquete `probability`. Esta representación trabaja con probabilidades racionales, lo que facilita comparar salidas de forma exacta durante la validación. Para evitar diferencias superficiales entre representaciones equivalentes de una misma distribución, los programas de prueba normalizan sus resultados mediante `Dist.norm` antes de imprimirlos y compararlos.

Aunque el ejemplo guía del capítulo Problema usa `Maybe`, el dominio de interés es probabilístico. `Maybe` actúa como una mónada pequeña y familiar para explicar el reordenamiento; `Dist.T Rational` permite evaluar la misma idea en programas que efectivamente representan distribuciones.

2.5. ApplicativeDo

`ApplicativeDo` es una extensión de GHC basada en el trabajo de Marlow, Peyton Jones, Kmett y Mokhov [13]. Su objetivo es permitir que programas escritos con *do-notation* usen combinadores aplicativos cuando las dependencias lo permiten. De esta forma, el programador conserva la sintaxis monádica habitual, mientras el compilador puede generar una estructura de ejecución más estática.

El proceso conceptual descrito por Marlow et al. tiene dos etapas. Primero, la etapa de *rearrangement* analiza la lista de statements de un bloque `do` y construye un plan que combina secuencias monádicas con grupos applicative. Luego, la etapa de *desugaring* traduce ese plan a expresiones con operaciones como `fmap`, `pure`, `(<*>)`, `join` y `(>>=)`.

La Figura 2.1 resume la etapa de *rearrangement*. Esta etapa busca cortes válidos en la secuencia de statements, de modo que partes independientes puedan agruparse applicativamente. La validez de un corte depende de que las variables definidas en una parte no sean requeridas por la otra de forma incompatible con el alcance del programa.

La Figura 2.2 muestra la segunda etapa. Una vez construido el plan, el compilador reemplaza la notación de alto nivel por combinadores explícitos. Así, un bloque que originalmente

$$\begin{aligned}
\text{rearrange } \{s_1; \dots; s_n\} &= \{s_1\}, & \text{if } n = 1 \\
&= \text{split } g_1, & \text{if } k = 1 \\
&= \{(\text{split } g_1 \mid \dots \mid \text{split } g_k)\}, & \text{otherwise}
\end{aligned}$$

where

$$g_1 \dots g_k = \text{segments } \{s_1; \dots; s_n\}$$

$$\text{segments } \{s_1; \dots; s_n\} = \{s_1; \dots; s_{i_1}\} \dots \{s_{(i_k)+1}; \dots; s_n\}$$

where

$$\begin{aligned}
i_1 \dots i_k &= \{ i \in 1 \dots n \\
&\quad \mid \text{bv}\{s_1; \dots; s_i\} \cap \text{fv}\{s_{i+1}; \dots; s_n\} = \emptyset \}
\end{aligned}$$

$$\begin{aligned}
\text{split}\{s_1; \dots; s_n\} &= \{s_1\}, & \text{if } n = 1 \\
&= \text{splitat } i_{\text{opt}}, & \text{otherwise}
\end{aligned}$$

where

$$\text{splitat } i = \text{rearrange } \{s_1; \dots; s_i\} ++ \text{rearrange } \{s_{i+1}; \dots; s_n\}$$

$i_{\text{opt}} \in 1 \dots n$ **such that**

$$\forall j. 1 \leq j < n. \text{cost}_s(\text{splitat } j) \geq \text{cost}_s(\text{splitat } i_{\text{opt}})$$

$$\text{cost}_s \{s_1; \dots; s_n\} = \Sigma\{\text{cost}_a s_i \mid 1 \leq i \leq n\}$$

$$\text{cost}_a (\mathbf{p} \leftarrow \mathbf{e}) = 1$$

$$\text{cost}_a (l_1 \mid \dots \mid l_n) = \max\{\text{cost}_s l_i \mid 1 \leq i \leq n\}$$

$$\text{fv } \{s_1; \dots; s_n\} = \text{the free variables of } s_1 \dots s_n$$

$$\text{bv } \{s_1; \dots; s_n\} = \text{the variables bound by } s_1 \dots s_n$$

Figura 2.1: Etapa de *rearrangement* propuesta por Marlow et al.

parece puramente monádico puede terminar expresado parcialmente en términos aplicativos.

El modelo de costos asociado distingue entre secuencias y grupos aplicativos. Un statement elemental tiene costo unitario; una secuencia suma los costos de sus componentes; y un grupo aplicativo toma el máximo costo entre sus ramas. Por lo tanto, exponer independencia puede reducir el costo abstracto del plan, incluso si el programa fuente se mantiene escrito con `do`.

La limitación relevante para esta memoria es que **ApplicativeDo** preserva el orden relativo original de los statements. Esto es necesario para mónadas arbitrarias, pero impide considerar planes que requieren mover statements independientes antes de aplicar el algoritmo. El aporte de esta memoria se ubica exactamente en esa brecha: no reemplaza **ApplicativeDo**, sino que le entrega órdenes alternativos cuando el bloque fue marcado como conmutativo.

Con esto queda establecida la brecha que desarrolla el siguiente capítulo: **ApplicativeDo** mejora planes al introducir estructura aplicativo, pero no explora permutaciones del orden

$$\text{desugar} :: \text{Stmts} \rightarrow \text{Expr} \rightarrow \text{Expr}$$

$$\text{desugar } \{\} e = \text{pure } e \quad (0)$$

$$\begin{aligned} \text{desugar } \{p \leftarrow e\} e' & \quad (1) \\ \mid p == e' & = e \\ \mid \text{otherwise} & = (\lambda p \rightarrow e') <\$> e \end{aligned}$$

$$\text{desugar } \{p \leftarrow e; l\} e' = e >>= \lambda p \rightarrow \text{desugar } l e' \quad (2)$$

$$\begin{aligned} \text{desugar } \{(l_1 \mid \dots \mid l_n)\} e & \quad (3) \\ = (\lambda p_1 \dots p_n \rightarrow e) <\$> e_1 <*> \dots <*> e_n \\ \text{where } (p_i, e_i) & = \text{desugar}_{arg} l_i \text{fv}(e) \end{aligned}$$

$$\begin{aligned} \text{desugar } \{(l_1 \mid \dots \mid l_n); s\} e & \quad (4) \\ = \text{join } ((\lambda p_1 \dots p_n \rightarrow e') <\$> e_1 <*> \dots <*> e_n) \\ \text{where} & \\ e' = \text{desugar } s e & \\ (p_i, e_i) = \text{desugar}_{arg} l_i \text{fv}(e') & \end{aligned}$$

$$\begin{aligned} \text{desugar}_{arg} & :: \text{Stmts} \rightarrow \text{Set Var} \rightarrow (\text{Pat}, \text{Expr}) \\ \text{desugar}_{arg} \{p \leftarrow e\} vs & = (p, e) \\ \text{desugar}_{arg} l vs & = ((v_1, \dots, v_k), \text{desugar } l (v_1, \dots, v_k)) \\ \text{where} & \\ v_1 \dots v_k & = \text{bv}(l) \cap vs \end{aligned}$$

Figura 2.2: Etapa de *desugaring* propuesta por Marlow et al.

escrito por el programador.

Capítulo 3

Problema

El problema se ilustrará con un caso pequeño sobre `Maybe`. La misma idea se evalúa luego sobre `Dist.T Rational`, pero `Maybe` permite aislar el fenómeno de ordenamiento sin introducir detalles propios de distribuciones probabilísticas.

```
mainExample :: Maybe (Int, Int, Int, Int)
mainExample = CD.do
  x1 <- Just 1
  x2 <- Just (x1 + 10)
  x3 <- Just 100
  x4 <- Just (x1 + x3)
  CD.return (x1, x2, x3, x4)
```

Para el análisis, se nombran los cuatro statements previos al `return` como `s1`, `s2`, `s3` y `s4`. El orden original es `s1 ; s2 ; s3 ; s4`. En ese orden, `s3` no depende de `s2`, pero aparece después de él; esa ubicación impide que `ApplicativeDo` exponga el mejor plan disponible bajo conmutatividad.

3.1. Limitación del Orden Sintáctico

La implementación estándar de `ApplicativeDo` transforma bloques `do` preservando el orden relativo de los statements escritos por el programador. Esta restricción es correcta para mónadas arbitrarias: si un bloque contiene efectos observables, como escrituras por pantalla, lectura de archivos o mutación de estado, intercambiar dos acciones puede cambiar el comportamiento del programa.

El costo de esa seguridad es que el algoritmo solo puede descubrir paralelismo cuando este aparece compatible con el orden sintáctico original. Si dos subcómputos independientes no están ubicados en posiciones convenientes, `ApplicativeDo` puede generar un plan mejor que la ejecución secuencial, pero no necesariamente el mejor plan disponible bajo una hipótesis

más fuerte. En particular, para una mónada conmutativa, ciertos statements independientes pueden cambiar de posición sin alterar la semántica. Esa posibilidad no está explotada por el algoritmo base de Marlow et al., porque su diseño debe seguir siendo válido para cualquier mónada [13].

La limitación estudiada en esta memoria aparece precisamente en esa diferencia: preservar el orden es necesario en el caso general, pero puede ser demasiado conservador cuando el programa declara explícitamente que el bloque se evalúa en una mónada conmutativa. El problema consiste entonces en determinar qué reordenamientos son semánticamente admisibles y si alguno de ellos permite que `ApplicativeDo` construya un plan de menor costo.

3.2. Ejecución Secuencial

El punto de comparación más simple es ejecutar el bloque en el orden monádico original, sin aprovechar `ApplicativeDo`. En ese escenario, cada statement debe esperar la finalización del anterior, aun cuando no exista una dependencia directa entre ellos.

Con el modelo de costos usado por `ApplicativeDo`, cada statement elemental tiene costo unitario. Por lo tanto, el plan secuencial del ejemplo es:

```
s1 ; s2 ; s3 ; s4
```

Su costo total es la suma de los cuatro statements:

$$1 + 1 + 1 + 1 = 4$$

Este valor fija el baseline del capítulo. Las transformaciones posteriores se comparan contra este costo para mostrar primero la mejora obtenida por `ApplicativeDo` y luego la mejora adicional obtenida al permitir reordenamiento conmutativo.

3.3. ApplicativeDo Actual

Al activar `ApplicativeDo`, GHC puede reemplazar parte del comportamiento monádico por combinadores aplicativos cuando las dependencias lo permiten. Para el orden original del ejemplo, el algoritmo puede construir el siguiente plan:

```
s1 ; (s2 | (s3 ; s4))
```

La notación `|` indica composición applicative, mientras que `;` indica dependencia secuencial. En este plan, `s1` debe ejecutarse primero porque produce `x1`, usado por `s2` y `s4`. Luego,

s2 puede ejecutarse en paralelo con la secuencia formada por **s3** y **s4**. Como **s4** necesita **x3**, **s4** debe quedar después de **s3** dentro de esa rama.

El costo del plan es:

$$1 + \text{máx}(1, 1 + 1) = 3$$

Esto muestra que **ApplicativeDo** ya mejora el baseline secuencial. Sin embargo, el resultado todavía depende del orden textual original. El statement **s3** es independiente de **s2**, pero aparece después de él; como el algoritmo estándar no reordena statements, no puede evaluar el plan que resultaría de adelantar **s3**. Esta es la brecha que aborda la extensión propuesta.

3.4. Plan con Reordenamiento

Si se permite reordenar statements bajo la hipótesis de conmutatividad, las dependencias del ejemplo admiten la permutación $[1, 3, 2, 4]$. En términos de statements, esta permutación corresponde a:

s1 ; **s3** ; **s2** ; **s4**

El reordenamiento es válido porque conserva las dependencias necesarias: **s1** sigue antes de **s2**, **s1** sigue antes de **s4** y **s3** sigue antes de **s4**. Al entregar este nuevo orden al algoritmo de **ApplicativeDo**, el plan resultante es:

$(\text{s1} \mid \text{s3}) ; (\text{s2} \mid \text{s4})$

Su costo es:

$$\text{máx}(1, 1) + \text{máx}(1, 1) = 2$$

Este plan expresa la mejora central de la memoria. Primero se ejecutan en paralelo **s1** y **s3**; luego, una vez disponibles **x1** y **x3**, se ejecutan en paralelo **s2** y **s4**. El algoritmo de **ApplicativeDo** no fue reemplazado: la extensión amplía su entrada con órdenes semánticamente admisibles y selecciona el candidato de menor costo.

3.5. Formulación

El problema general abordado por esta memoria puede formularse de la siguiente manera. Dado un bloque **do** marcado explícitamente como conmutativo, se desea ampliar el espacio de

órdenes que **ApplicativeDo** puede evaluar, sin permitir órdenes que rompan dependencias locales entre statements. Cada orden admisible produce un plan de ejecución candidato, y el compilador debe seleccionar el de menor costo según el modelo adoptado.

El alcance semántico de esta formulación depende de una hipótesis explícita: el programador declara que la mónada del bloque se tratará como conmutativa. El compilador no demuestra esa propiedad. En cambio, usa la marca sintáctica como un contrato de uso: si el bloque está marcado, entonces se permite explorar intercambios que respeten dependencias de variables; si no está marcado, se conserva el comportamiento estándar de **ApplicativeDo**.

Por lo tanto, la contribución no consiste en reemplazar **ApplicativeDo**, sino en extender su espacio de búsqueda bajo una condición adicional. En el ejemplo usado en este capítulo, esa extensión transforma la comparación de costos desde 4 contra 3 a una comparación que también incluye el plan de costo 2, obtenido por una permutación semánticamente válida. La construcción formal de esas permutaciones se presenta en el capítulo de Solución.

Capítulo 4

Solución

4.1. Visión General

La solución extiende experimentalmente el flujo de `ApplicativeDo` en `GHC`. La idea central es mantener el algoritmo existente de `ApplicativeDo`, pero entregarle más de un orden posible de statements cuando el bloque fue marcado explícitamente como conmutativo. Cada orden alternativo debe respetar las dependencias RAW locales; luego, el compilador evalúa el costo del plan applicative resultante y selecciona el candidato más barato.

El pipeline general es el siguiente:

```
CD.do
-> Renamer
-> detección del marcador conmutativo
-> construcción del grafo RAW
-> enumeración de ordenamientos topológicos
-> evaluación con ApplicativeDo
-> selección por costo mínimo
-> generación de ApplicativeStmt
```

La extensión solo cambia el espacio de búsqueda cuando el bloque usa la notación conmutativa experimental. Si el programa usa `do` normal, o si el módulo calificador no exporta el marcador esperado, el compilador conserva el comportamiento estándar: se evalúa únicamente el orden sintáctico original.

Esta decisión permite separar dos responsabilidades. Por un lado, el programador afirma que el bloque puede tratarse bajo una hipótesis de conmutatividad. Por otro lado, el compilador garantiza que los reordenamientos considerados no violen dependencias locales de variables y que el plan final siga pasando por el mecanismo normal de `ApplicativeDo`.

4.2. QualifiedDo

La activación del reordenamiento no se realiza mediante una flag global, sino mediante sintaxis explícita de `QualifiedDo`. El programador importa el módulo conmutativo experimental y escribe el bloque usando `CD.do`:

```
{-# LANGUAGE ApplicativeDo #-}
{-# LANGUAGE QualifiedDo #-}

import qualified Control.Monad.CommutativeDo as CD

example = CD.do
  x <- Just 1
  y <- Just 2
  CD.return (x, y)
```

El módulo público `Control.Monad.CommutativeDo` reexporta las operaciones necesarias para `QualifiedDo`: `bind`, secuenciación, `return`, `pure`, `fmap`, aplicación `applicative`, `join` y `fail`. Además, exporta el marcador `--commutative-do--`. Durante el Renamer, `GHC` revisa si el módulo calificador del bloque exporta ese marcador; si lo encuentra, el bloque queda habilitado para la enumeración de permutaciones.

La clase `CommutativeMonad` cumple un rol declarativo. No constituye una prueba formal de conmutatividad dentro del compilador, sino una afirmación del programador sobre el uso previsto de la mónada. En consecuencia, la extensión asume esa propiedad solo cuando el bloque fue escrito con la notación `opt-in`.

La flag de selección de candidatos, presentada más adelante, no activa el reordenamiento. Solo permite escoger un candidato ya generado. La activación sigue dependiendo de `CD.do`, del marcador conmutativo y de que `ApplicativeDo` esté habilitado.

4.3. Información del Renamer

La implementación se ubica en el Renamer, antes del typechecker y del desugarer. Esta etapa resulta conveniente porque los statements del bloque `do` ya fueron renombrados y el compilador conserva información sobre variables libres. En particular, el flujo de `HsDo` usa `rnStmtsWithFreeVars` para renombrar los statements y entregar, junto a cada uno, el conjunto de nombres libres que aparecen en su cuerpo.

La extensión aprovecha dos fuentes de información. La primera son las variables libres de cada statement, que permiten saber qué nombres son leídos. La segunda son los binders definidos por el statement, obtenidos mediante la utilidad `collectStmtBinders`. Con ambas se calculan lecturas locales y escrituras locales dentro del bloque.

Esta decisión evita introducir un análisis posterior sobre Core o sobre una representación desazucarada. El reordenamiento ocurre antes de que **ApplicativeDo** convierta el bloque en **ApplicativeStmt**, de modo que el algoritmo todavía puede trabajar directamente sobre la lista de statements de la *do-notation*.

Además, al trabajar después del renombrado, las variables locales se representan mediante nombres únicos. Esto simplifica el modelo de dependencias usado por la implementación: la restricción central es que toda lectura local debe quedar después de la escritura que introduce ese nombre.

4.4. Dependencias Def-Use y Grafo RAW

El reordenamiento de statements debe preservar las relaciones *def-use*: si un statement usa una variable local, la definición de esa variable debe mantenerse antes. La implementación captura esta restricción mediante un grafo RAW, abreviatura de *Read After Write*. Sus nodos son los statements del bloque, indexados en el orden original desde 1. Sus aristas representan dependencias de lectura después de escritura entre variables locales.

Para cada statement **s**, la implementación calcula:

- **WRITE(s)**: conjunto de nombres locales definidos por **s**.
- **READ_local(s)**: variables libres de **s** restringidas al conjunto de nombres definidos dentro del bloque.

Luego, existe una arista desde **si** hacia **sj** si el statement **sj** lee una variable escrita por **si**. En notación de conjuntos:

$$\text{WRITE}(\text{si}) \cap \text{READ_local}(\text{sj}) \neq \emptyset$$

En el ejemplo guía, **s1** define **x1**, y **s2** y **s4** leen **x1**; por eso aparecen las aristas **s1** → **s2** y **s1** → **s4**. A su vez, **s3** define **x3**, usado por **s4**; por eso aparece la arista **s3** → **s4**.

Statement	WRITE	READ_local
s1	x1	$\emptyset$
s2	x2	x1
s3	x3	$\emptyset$
s4	x4	x1, x3

El grafo resultante puede representarse así:

s1 -----> **s2**

```

|
+-----> s4
  ^
s3 -----+

```

Este grafo distingue permutaciones válidas de permutaciones inválidas. Por ejemplo, `s1 ; s3 ; s2 ; s4` respeta todas las aristas, mientras que `s3 ; s4 ; s1 ; s2` no es válida, porque ubica `s4` antes de `s1`, aun cuando `s4` necesita el valor `x1` definido por `s1`.

El prototipo general del proyecto consideró también dependencias WAR y WAW como antecedente para modelos de asignaciones imperativas. La implementación vigente dentro del Renamer no las usa como restricciones internas reales. Después del renombrado, los binders locales son nombres únicos, por lo que no hay sobrescritura del mismo nombre renombrado dentro del bloque. En este contexto, preservar RAW captura la restricción def-use necesaria para generar órdenes alternativos bien formados.

4.5. Permutaciones

Una vez construido el grafo RAW, la extensión enumera ordenamientos topológicos del grafo. Cada ordenamiento topológico corresponde a una secuencia de statements que respeta todas las aristas de precedencia. Por lo tanto, cada uno representa una forma alternativa de escribir el bloque sin violar dependencias locales.

Para el ejemplo guía, el grafo tiene cinco ordenamientos válidos:

Candidato	Orden de statements
0	[1, 2, 3, 4]
1	[1, 3, 2, 4]
2	[1, 3, 4, 2]
3	[3, 1, 2, 4]
4	[3, 1, 4, 2]

El candidato 0 corresponde al orden original. Los demás candidatos mueven `s3` o `s1` sin violar las aristas RAW. Por ejemplo, `[1, 3, 2, 4]` es válido porque `s1` sigue antes de `s2` y `s4`, y `s3` sigue antes de `s4`.

La enumeración de ordenamientos topológicos puede crecer rápidamente con la cantidad de statements independientes. Por ello, la extensión se presenta como experimental: permite estudiar el espacio de planes que se abre bajo conmutatividad, pero la explosión combinatoria debe considerarse una limitación y una línea de trabajo futuro.

4.6. Evaluación de Candidatos

Cada permutación válida se entrega al algoritmo existente de **ApplicativeDo**. En otras palabras, la extensión no define un nuevo algoritmo de construcción de planes applicative; reutiliza el mecanismo de **GHC** sobre distintas secuencias de entrada. El resultado de cada evaluación es un árbol de statements junto con su costo.

En el ejemplo guía, la evaluación produce la siguiente comparación:

Candidato	Orden	Plan conceptual	Costo
0	[1,2,3,4]	$s_1; (s_2 \mid (s_3; s_4))$	3
1	[1,3,2,4]	$(s_1 \mid s_3); (s_2 \mid s_4)$	2
2	[1,3,4,2]	$(s_1 \mid s_3); (s_4 \mid s_2)$	2
3	[3,1,2,4]	$(s_3 \mid s_1); (s_2 \mid s_4)$	2
4	[3,1,4,2]	$(s_3 \mid s_1); (s_4 \mid s_2)$	2

El candidato original alcanza costo 3, que coincide con **ApplicativeDo** normal sobre el orden escrito por el programador. Los candidatos restantes alcanzan costo 2 porque exponen dos niveles de paralelismo applicative: primero entre los productores independientes y luego entre los consumidores que ya tienen disponibles sus entradas.

Esta evaluación es el punto donde se integra la contribución con el estado del arte. La extensión amplía el conjunto de órdenes posibles; **ApplicativeDo** sigue siendo el componente que decide cómo agrupar cada orden y cuánto cuesta el plan resultante.

4.7. Selección

Después de evaluar todas las permutaciones válidas, el compilador debe seleccionar un único candidato final. La política automática implementada es escoger el candidato de menor costo. Si varios candidatos empatan con el mismo costo mínimo, se conserva el primero en el orden de enumeración.

En el ejemplo guía, el candidato 0 tiene costo 3 y los candidatos 1, 2, 3 y 4 tienen costo 2. Por lo tanto, la selección automática elige el candidato 1:

```
candidate-index = 1
index-order     = [1,3,2,4]
minimum-cost    = 2
```

Los índices de candidatos parten desde 0, porque se usan para seleccionar una permutación generada dentro del conjunto de candidatos. En cambio, los índices de statements parten desde 1, porque identifican la posición original de cada statement dentro del bloque **do**.

Esta separación es importante para leer las trazas: un `candidate-index` identifica una permutación en la lista generada, mientras que un `index-order` describe el orden de statements originales que conforma esa permutación.

4.8. Integración en GHC

La implementación principal se integra en **GHC**, no como una herramienta externa de reescritura. El cambio central está en el módulo `GHC.Rename.Expr`, específicamente en el flujo que procesa expresiones `HsDo` y llama a `postProcessStmtsForApplicativeDo`. Allí, después de renombrar los statements, la extensión puede decidir si corresponde ejecutar el reordenamiento conmutativo.

Los componentes principales de la integración son:

- `GHC.Rename.Expr`: contiene la detección del bloque conmutativo, construcción del grafo, enumeración de permutaciones, evaluación de candidatos y selección final.
- `GHC.Internal.Control.Monad.CommutativeDo`: define el módulo interno con el marcador conmutativo y las operaciones para `QualifiedDo`.
- `Control.Monad.CommutativeDo`: expone el módulo público usado por los experimentos.
- `GHC.Driver.DynFlags` y `GHC.Driver.Session`: registran la flag experimental de selección de candidato.

Las rutas completas se mantienen documentadas en el repositorio y en los anexos técnicos; en el cuerpo principal basta con identificar los módulos y su responsabilidad.

El resultado final vuelve al flujo normal de `ApplicativeDo`. Una vez elegido el árbol de statements, **GHC** reconstruye los statements finales y produce `ApplicativeStmt` cuando el plan lo permite. Los detalles más densos de funciones internas y trazas se reservan para los anexos y la documentación técnica del repositorio.

4.9. Selección Forzada

Para validar semánticamente la extensión no basta con ejecutar el candidato seleccionado automáticamente. También es necesario compilar y ejecutar cada permutación válida, de modo que todas las variantes generadas puedan compararse contra el resultado esperado. Para ello se agregó una flag experimental:

```
-fado-reorder-candidate-n=<n>
```

Cuando la flag recibe un índice válido, el Renamer selecciona el candidato `n` en vez del candidato de menor costo. Si el índice es negativo, está fuera de rango o no se entregó la flag, se usa la selección automática por costo mínimo.

Esta flag no activa el reordenamiento. La generación de candidatos sigue dependiendo de que el bloque use `CD.do`, de que el módulo calificador exporte el marcador conmutativo y de que `ApplicativeDo` esté habilitado. La flag solo cambia cuál de los candidatos ya generados se elige como resultado final del compilador.

El pipeline de validación usa esta capacidad para construir binarios `permutation.i`. Luego ejecuta cada binario y compara su salida con la variante óptima. Si todas las permutaciones válidas producen la misma salida observable, la evidencia experimental apoya que el reordenamiento preservó la semántica en ese caso.

Capítulo 5

Validación

5.1. Metodología

La validación busca comprobar dos propiedades experimentales. Primero, que las permutaciones generadas por la extensión preserven la salida observable de los programas evaluados. Segundo, que el reordenamiento pueda producir planes con menor costo abstracto que `ApplicativeDo` aplicado únicamente al orden original.

El pipeline de validación compila varias variantes de cada caso:

- el programa original;
- el programa con `ApplicativeDo` normal, sin marcador conmutativo;
- el candidato óptimo seleccionado automáticamente por la extensión;
- cada permutación válida, forzada mediante la flag de selección de candidato.

Luego, el pipeline ejecuta cada binario, captura `stdout`, registra el código de salida y compara los resultados contra la referencia. En los casos sintéticos, una validación exitosa exige que las salidas y códigos de salida coincidan. En los casos probabilísticos, los programas imprimen distribuciones normalizadas para evitar diferencias superficiales de representación.

Existen dos modos de ejecución. Los casos que no dependen de paquetes externos pueden compilarse directamente con el `GHC` modificado. Los casos probabilísticos se validan mediante `Cabal`, porque usan el paquete `probability`. En ambos modos, el resultado se guarda bajo `tests/`, junto con trazas del `Renamer`, resúmenes de costo y outputs de ejecución.

5.2. Artefactos de Validación

La validación deja artefactos reproducibles para inspeccionar el comportamiento del compilador y respaldar los resultados. Los experimentos viven bajo `experiments/`, mientras que

los resultados se guardan bajo `tests/` siguiendo una estructura paralela por mónada, familia, caso y variante.

Los archivos más relevantes son:

Artefacto	Función
<code>summary.log</code>	Resume costos y cantidad de permutaciones.
<code>semantic-validation.log</code>	Registra compilaciones, ejecuciones y comparación final.
<code>logs/raw.log</code>	Guarda la traza cruda del Renamer.
<code>logs/graph.log</code>	Presenta el grafo RAW en forma legible.
<code>logs/optimal-reorder.log</code>	Muestra el candidato elegido automáticamente.
<code>logs/original_ado.log</code>	Muestra <code>ApplicativeDo</code> normal, sin reordenamiento.
<code>logs/permutation_i.log</code>	Muestra el candidato <code>i</code> forzado por flag.

Estos artefactos cumplen dos roles. Primero, permiten depurar la extensión observando el grafo, las permutaciones y los árboles seleccionados. Segundo, entregan la base empírica para comparar costos y salidas entre variantes.

5.3. Métricas

La comparación de planes usa métricas emitidas por las trazas del Renamer. Las claves principales son:

```
original-cost
applicative-do-cost
reorder-and-ado-minimum-cost
generated-permutations
minimum-cost-permutations
```

La primera métrica corresponde al costo secuencial base del bloque. La segunda mide el costo obtenido por `ApplicativeDo` sobre el orden original. La tercera mide el menor costo encontrado al evaluar las permutaciones válidas. Las dos últimas indican cuántos candidatos fueron generados y cuántos de ellos empatan en el costo mínimo.

El costo es abstracto, no una medición de tiempo real. En el modelo actual, cada `statement` elemental tiene costo unitario; una secuencia suma costos; y una composición `applicative` toma el máximo costo de sus ramas. Por ello, la métrica sirve para comparar la forma del plan de ejecución, no para afirmar una aceleración empírica directa en tiempo de pared.

Además de las métricas de costo, la validación registra el resultado semántico de cada caso. Un caso exitoso termina con `RESULT: OK`; un control negativo diseñado para mostrar cambio observable puede terminar con `RESULT: FAIL`, como ocurre con el caso de `IO`.

5.4. Corpus Sintético Maybe

El primer corpus sintético usa la mónada `Maybe`. Su objetivo es validar fenómenos estructurales del Renamer y del grafo de precedencia con programas pequeños, deterministas y fáciles de comparar. La descripción completa de los casos está en el archivo `cases.md` del corpus `maybe-monad`.

Las familias implementadas cubren los siguientes aspectos:

- **dependency-shapes**: formas de grafo como cadenas, grafo vacío, diamante, fanout, fanin y cadenas independientes.
- **binders**: patrones que definen múltiples variables, patrones de lista, patrones anidados, wildcards, as-patterns y patrones lazy.
- **let-statements**: interacción entre statements `let`, binders monádicos y shadowing.
- **body-stmts**: statements sin binder, tanto independientes como dependientes.
- **maybe-failure**: comportamiento de `Nothing`, fallos de patrón y guards.
- **shadowing**: reutilización del mismo nombre fuente con nombres renombrados distintos.
- **cost-selection**: casos donde el orden original no es mínimo, todos los costos empatan o existe un mínimo único.
- **controls**: controles negativos y positivos para verificar que el marcador conmutativo activa la extensión solo cuando corresponde.

Este corpus permite aislar errores en la recolección de binders, el cálculo de variables libres locales, la construcción de aristas RAW y la selección de candidatos. El caso guía del informe pertenece a la familia **cost-selection**: su resumen reporta costo secuencial 4, costo con `ApplicativeDo` 3 y costo mínimo con reordenamiento 2.

5.5. Corpus Sintético Probabilístico

El segundo corpus sintético usa la mónada probabilística `Dist.T Rational` del paquete `probability`. Este corpus conecta la extensión con el dominio principal de la memoria: programas probabilísticos finitos escritos en `Haskell`. Su descripción completa está en el archivo `cases.md` del corpus `probability-monad`.

Los programas probabilísticos imprimen una salida canónica usando `Dist.norm`. Esta normalización agrupa resultados iguales y suma sus probabilidades, evitando que dos representaciones internas equivalentes se comparen como diferentes por el pipeline. La comparación sigue siendo byte a byte, pero sobre una representación normalizada de la distribución.

El corpus reutiliza varias familias estructurales de `Maybe`, como formas de dependencia, binders, `let`, `BodyStmt`, shadowing, selección por costo y controles. Además, incorpora familias específicas de probabilidades:

- distribuciones independientes con pesos no uniformes;
- resultados duplicados que deben sumarse al normalizar;
- soportes dependientes de valores generados previamente;
- pesos dependientes de binders anteriores;
- distribuciones construidas con `Dist.choose`, `Dist.relative` o `Dist.fromFreqs`;
- condicionamiento explícito mediante distribuciones imposibles o filtrado.

Esta separación permite validar dos dimensiones: que el Renamer se comporte igual que en el corpus estructural y que la semántica probabilística observable se conserve después del reordenamiento.

5.6. Corpus Real

La validación con corpus real forma parte del plan de evaluación de la memoria. No se presenta como trabajo futuro, sino como una sección de resultados que debe completarse cuando se seleccionen, adapten y ejecuten programas probabilísticos externos o menos sintéticos.

Los criterios de selección son los siguientes:

- programas escritos en `Haskell` con bloques `do` probabilísticos;
- presencia de dependencias no triviales entre statements;
- existencia de subcómputos potencialmente independientes;
- salida determinista y comparable por el pipeline;
- tamaño de soporte suficientemente pequeño para ejecutar la validación completa.

El protocolo de adaptación debe documentar cada cambio realizado: reemplazo de `do` por `CD.do`, incorporación de la instancia conmutativa si corresponde, normalización de la salida y simplificaciones necesarias para ejecutar el caso en el entorno reproducible. Para cada programa real se deben reportar las mismas métricas que en los corpus sintéticos: costos, cantidad de permutaciones, candidato seleccionado y resultado semántico.

Esta sección permitirá evaluar aplicabilidad externa. Los corpus sintéticos muestran que la extensión funciona sobre fenómenos controlados; el corpus real debe mostrar si esos fenómenos aparecen en programas representativos y si el costo del espacio de permutaciones sigue siendo manejable.

5.7. Control IO

El control con `IO` muestra por qué el reordenamiento no puede activarse para mónadas arbitrarias. En `IO`, dos acciones que no comparten variables pueden tener efectos observables cuyo orden importa. Por ejemplo, dos impresiones independientes pueden producir salidas distintas si se intercambian.

El log de validación del control negativo registra dos permutaciones. La permutación original imprime:

```
A
B
3
```

Una permutación alternativa imprime:

```
B
A
3
```

La comparación semántica detecta la diferencia y termina con **RESULT: FAIL**. Este resultado no invalida la solución; al contrario, confirma la necesidad del mecanismo de opt-in. El compilador no debe asumir conmutatividad por ausencia de dependencias RAW, porque los efectos observables pueden seguir dependiendo del orden.

Por esta razón, la extensión solo explora reordenamientos cuando el bloque fue marcado explícitamente como conmutativo mediante `CD.do`. Para bloques `do` normales, `ApplicativeDo` conserva el comportamiento estándar y no se habilita la enumeración de permutaciones conmutativas.

5.8. Resultados

Los resultados completos se obtienen desde los logs de resumen y de validación semántica generados para cada caso. En el estado actual del repositorio, los corpus sintéticos de `Maybe` y de `Dist.T Rational` están implementados y testeados; la tabla agregada final debe consolidar esos registros por familia y variante.

Como resultado representativo, el caso guía de `Maybe` reporta:

Caso	Sec.	ADo	Reord.	Perms.	Mínimas
Ejemplo guía <code>Maybe</code>	4	3	2	5	4

La tabla muestra la mejora estricta que motiva la memoria: el costo secuencial es 4, `ApplicativeDo` sobre el orden original reduce el costo a 3, y el reordenamiento conmutativo encuentra un plan de costo 2. Además, el caso genera cinco permutaciones válidas, de las cuales cuatro alcanzan el costo mínimo.

Para el corpus probabilístico, la tabla final debe reportar los mismos campos, junto con el resultado semántico de cada caso. La comparación debe realizarse sobre distribuciones normalizadas con `Dist.norm`. Para el corpus real, esta sección debe incorporar los resultados una vez que los programas hayan sido seleccionados y ejecutados con el mismo protocolo.

5.9. Amenazas a la Validez

La evidencia experimental tiene varias amenazas a la validez que deben considerarse al interpretar los resultados.

En primer lugar, una parte importante del corpus es sintética. Esto permite controlar formas de dependencia, patrones de binders y casos de costo, pero no garantiza por sí solo que las mismas oportunidades de reordenamiento aparezcan con frecuencia en programas reales. Por esa razón, el corpus real queda definido como parte de la validación en progreso.

En segundo lugar, la validación semántica compara salidas observables. Esta estrategia es adecuada para los casos finitos evaluados, pero no constituye una prueba formal de preservación semántica para todo programa posible. En el corpus probabilístico, la normalización con `Dist.norm` reduce diferencias representacionales, pero también fija una convención de comparación que debe mantenerse explícita.

En tercer lugar, la enumeración de permutaciones puede crecer de forma combinatoria. Los casos pequeños permiten inspección exhaustiva, pero programas con muchos statements independientes podrían generar un espacio de búsqueda grande. Esta limitación afecta tanto el costo de compilación como la viabilidad de validar todos los candidatos.

Finalmente, la conmutatividad no es demostrada por `GHC`. El mecanismo de `CD.do` representa una afirmación del programador. Si se marca como conmutativo un bloque cuya mónada no preserva semántica bajo intercambio de efectos independientes, el compilador puede generar un programa observablemente distinto. El control con `IO` ilustra este riesgo.

También debe destacarse que el modelo de costos es uniforme por statement. Este modelo permite comparar planes de manera simple y reproducible, pero no distingue operaciones baratas de operaciones costosas. Un refinamiento de costos anotados o inferidos queda fuera de la implementación actual.

Capítulo 6

Conclusión

6.1. Resumen del Trabajo

Esta memoria presentó una extensión experimental de `ApplicativeDo` en `GHC` para explorar reordenamientos de statements en bloques `do` marcados como conmutativos. El punto de partida fue una limitación del algoritmo existente: `ApplicativeDo` puede descubrir independencia y construir planes applicative, pero preserva el orden relativo original de los statements. Esta decisión es correcta para mónadas arbitrarias, aunque puede ser conservadora cuando la mónada permite conmutar subcómputos independientes.

La solución implementada agrega un mecanismo explícito de opt-in sintáctico. Los bloques escritos como `CD.do` se reconocen usando un marcador exportado por el módulo conmutativo experimental. Para esos bloques, el Renamer construye un grafo de precedencia basado en dependencias RAW, enumera ordenamientos topológicos válidos, evalúa cada orden con el algoritmo existente de `ApplicativeDo` y selecciona el candidato de menor costo.

La validación se apoyó en corpus sintéticos para `Maybe` y para la mónada probabilística `Dist.T Rational`, además de un control negativo con `IO`. El ejemplo guía mostró una mejora estricta de costo: ejecución secuencial de costo 4, `ApplicativeDo` normal de costo 3 y reordenamiento conmutativo de costo 2.

6.2. Cumplimiento de Objetivos

La Tabla siguiente resume el cumplimiento de los objetivos definidos en la Introducción.

Objetivo	Resultado
Caracterizar ApplicativeDo en GHC .	Se identificó el flujo del Renamer para HsDo , el punto donde se procesa ApplicativeDo y la construcción de statements aplicativos.
Diseñar un modelo de dependencias locales.	Se definió un grafo RAW usando variables definidas, variables libres y lecturas locales por statement.
Implementar reordenamiento conmutativo.	Se integró la generación de permutaciones topológicas y la evaluación de candidatos dentro del flujo de ApplicativeDo .
Agregar activación explícita y selección de candidatos.	Se agregó el módulo conmutativo público, el marcador de opt-in y la flag de selección forzada por índice.
Validar sobre corpus sintéticos.	Se implementaron y probaron corpus para Maybe y Dist.T Rational , con logs reproducibles de costos, grafos y salidas.
Definir validación con corpus real.	Se incorporó la sección y el protocolo de selección/adaptación dentro del capítulo Validación, pendiente de completar con programas reales.

En conjunto, los objetivos técnicos principales quedaron cubiertos por la modificación de **GHC**, los módulos agregados, los scripts de validación y la documentación del repositorio. La validación con corpus real queda ubicada dentro del plan experimental del informe, no como una extensión conceptual posterior al trabajo.

6.3. Limitaciones

La primera limitación es semántica: **GHC** no prueba que una mónada sea conmutativa. El mecanismo implementado se basa en una declaración explícita del programador mediante **CD.do**. Si esa declaración es incorrecta, el compilador puede aceptar reordenamientos que cambien efectos observables, como ilustra el control con **IO**.

La segunda limitación es combinatoria. Enumerar ordenamientos topológicos puede ser costoso cuando el bloque tiene muchos statements independientes. El prototipo permite estudiar exhaustivamente casos pequeños y medianos, pero requiere estrategias adicionales para escalar a programas de mayor tamaño.

La tercera limitación está en la validación. Comparar salidas observables entrega evidencia práctica para los casos evaluados, pero no equivale a una prueba formal de preservación semántica para todos los programas. En el corpus probabilístico, la normalización de distribuciones reduce diferencias representacionales, pero también fija una convención específica

de comparación.

Finalmente, el modelo de costos es uniforme por statement. Esta simplificación permite comparar estructura de planes de manera reproducible, pero no distingue entre statements baratos y costosos. Por lo tanto, el menor costo abstracto no necesariamente equivale al menor tiempo de ejecución real.

6.4. Trabajo Futuro

Una línea directa de trabajo futuro es refinar el modelo de costos. La implementación actual usa costo uniforme por statement, siguiendo un criterio simple y reproducible. Una extensión posterior podría permitir costos anotados o inferidos, por ejemplo `low`, `medium` y `high`, interpretados como costos 1, 2 y 3. Con ello, dos planes con el mismo número de niveles applicative podrían diferenciarse por el peso relativo de sus statements.

```
defaultExample :: Maybe Int
defaultExample = CD.do
  x1 <- Just 5           -- high
  x2 <- Just 10          -- low
  x3 <- Just (x1 + 15)   -- medium
  x4 <- Just (x2 + 20)   -- high
  CD.return (x3 + x4)
```

Otra línea es reducir el espacio de búsqueda. En lugar de enumerar todos los ordenamientos topológicos, el compilador podría usar heurísticas, límites configurables o búsqueda guiada por costo parcial. Esto permitiría aplicar la idea a bloques más grandes sin pagar siempre el costo de una enumeración exhaustiva.

También queda abierta una evaluación de rendimiento real. La memoria compara planes usando un costo abstracto heredado de `ApplicativeDo`; un trabajo posterior podría medir tiempos de compilación, tiempos de ejecución y efectos sobre backends o librerías que aprovechen paralelismo applicative.

Finalmente, sería valioso integrar casos de esta extensión en una testsuite más cercana a `GHC`, mejorar las trazas para inspección humana y avanzar hacia una formalización más completa de la preservación semántica bajo la hipótesis de conmutatividad. El corpus real, en cambio, no se lista aquí como trabajo futuro, porque forma parte de la validación definida en el capítulo anterior.

6.5. Cierre

El trabajo muestra una vía concreta para ampliar `ApplicativeDo` sin abandonar el compilador real ni la sintaxis habitual de `Haskell`. Bajo una hipótesis explícita de conmutatividad,

el orden escrito por el programador deja de ser el único orden que el compilador puede evaluar, siempre que se preserven las dependencias locales de variables.

El aporte principal no es solo el algoritmo de reordenamiento, sino su integración experimental en **GHC** con trazabilidad: módulos agregados, parches, corpus, scripts y logs reproducibles. Esto permite estudiar la idea de manera concreta, observar sus beneficios en casos controlados y delimitar sus riesgos.

En síntesis, la memoria demuestra que el espacio de planes de **ApplicativeDo** puede ampliarse para mónadas declaradas como conmutativas, obteniendo mejoras de costo en casos representativos sin modificar la semántica observable de los corpus evaluados.

Bibliografía

- [1] Esteban Romero Berríos. Repositorio experimental para el estudio de applicative-do y reordenamiento de statements en ghc, 2026. GitHub repository.
- [2] Alan L. Davis and Robert M. Keller. Data flow program graphs. *Computer*, 15(2):26–41, 1982.
- [3] Martin Erwig and Steve Kollmansberger. Functional pearls: Probabilistic functional programming in haskell. *Journal of Functional Programming*, 16(1):21–34, 2006.
- [4] Kai Feng, Jeremy Singer, and Angelos K. Marnierides. Fuzzrducc: Fuzzing with reconstructed def-use chain coverage, 2025.
- [5] GHC Contributors. Commit 8ecf6d8f7dfce9e5b1844cd196f83f00f3b6b879, 2015. GitHub commit.
- [6] GHC Contributors. Testsuite de la extensión applicivedo en ghc, 2026. Directorio `testsuite/tests/ado` del repositorio oficial de GHC.
- [7] The Glasgow Haskell Compiler Team. *GHC User’s Guide: Applicative do-notation*, 2026. Documentación oficial de GHC.
- [8] Mary Jean Harrold and Mary Lou Soffa. Efficient computation of interprocedural definition-use chains. *ACM Transactions on Programming Languages and Systems*, 16(2):175–204, 1994.
- [9] Bart Jacobs. From probability monads to commutative effectuses. *Journal of Logical and Algebraic Methods in Programming*, 94:200–237, 2018.
- [10] Anders Kock. Commutative monads as a theory of distributions. *Theory and Applications of Categories*, 26(4):97–131, 2012.
- [11] Simon Marlow. `ado`, 2015. GitHub repository.
- [12] Simon Marlow, Louis Brandy, Jonathan Coens, and Jon Purdy. There is no fork: An abstraction for efficient, concurrent, and concise data access. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming*, pages 325–337. ACM, 2014.
- [13] Simon Marlow, Simon Peyton Jones, Edward Kmett, and Andrey Mokhov. Desugaring Haskell’s do-notation into applicative operations. In *Proceedings of the 9th International Symposium on Haskell*, pages 92–104. ACM, 2016.

- [14] Simon Marlow, Ryan Newton, and Simon Peyton Jones. A monad for deterministic parallelism. In *Proceedings of the 4th ACM SIGPLAN Symposium on Haskell*, pages 71–82. ACM, 2011.
- [15] Conor McBride and Ross Paterson. Applicative programming with effects. *Journal of Functional Programming*, 18(1):1–13, 2008.
- [16] Andrey Mokhov, Georgy Lukyanov, Simon Marlow, and Jeremie Dimino. Selective applicative functors. *Proceedings of the ACM on Programming Languages*, 3(ICFP):1–29, 2019.
- [17] Justin Pombrio. *Resugaring: Lifting Evaluation Sequences through Syntactic Sugar*. PhD thesis, Brown University, 2018.
- [18] Adam Ścibior, Zoubin Ghahramani, and Andrew D. Gordon. Practical probabilistic programming with monads. In *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell*, pages 165–176. ACM, 2015.
- [19] Philip Wadler. Monads for functional programming. In *Advanced Functional Programming*, volume 925 of *Lecture Notes in Computer Science*, pages 24–52. Springer, 1995.
- [20] Brent Yorgey. The typeclassopedia. *The Monad.Reader*, 13:17–68, 2009.

Apéndice A

Anexos Técnicos

A.1. Reproducibilidad

Esta sección de anexo debe documentar el entorno reproducible. Existe para que el cuerpo principal no se sobrecargue con detalles de Dev Container, toolchain y build de GHC. Lo ideal es incluir comandos, versiones y referencias a `docs/toolchain.md`.

Para completarla, se deben listar versiones de GHC de bootstrap, Cabal, Alex, Happy y comandos Makefile relevantes. También conviene explicar la ruta esperada del binario modificado de GHC dentro del contenedor.

A.2. Pipeline de Validación

Esta sección de anexo debe describir con detalle los comandos usados para validar. Existe para que el capítulo Validación pueda concentrarse en metodología y resultados, dejando aquí la información operativa. Lo ideal es incluir ejemplos de `make test-ghc` y `make test-cabal`.

Para completarla, se debe documentar la estructura de entrada bajo `experiments/` y la estructura de salida bajo `tests/`. También conviene explicar la diferencia entre backend GHC directo y backend Cabal para dependencias externas como `probability`.

A.3. Logs y Trazas

Esta sección de anexo debe reunir ejemplos de logs representativos. Existe porque las trazas completas del Renamer son demasiado densas para el cuerpo principal, pero son importantes para reproducibilidad y depuración. Lo ideal es incluir fragmentos de `summary.log`, `optimal-reorder.log` y `semantic-validation.log`.

Para completarla, se deben seleccionar logs del ejemplo guía y de un caso probabilístico

representativo. También conviene explicar cómo leer `index-order`, `tree-cost` y el plan de ejecución con `|` y `;`.

A.4. Corpus

Esta sección de anexo debe contener tablas completas de los corpus. Existe para que Validación pueda presentar solo casos representativos sin perder trazabilidad del conjunto completo. Lo ideal es basarse en `experiments/maybe-monad/cases.md` y `experiments/probability-monad/cases.md`.

Para completarla, se deben copiar o sintetizar las tablas de familias, casos y variantes implementadas. Cuando el corpus real esté definido, también debe documentarse aquí con ruta, objetivo y estado de validación.

A.5. Detalles de Implementación

Esta sección de anexo debe contener los detalles de GHC que serían demasiado densos para el capítulo Solución. Existe para preservar trazabilidad técnica sin interrumpir la lectura principal. Lo ideal es referenciar `docs/renamer.md` y los parches relevantes.

Para completarla, se pueden listar funciones como `rearrangeForApplicativeDo`, `isCommutativeQuasi`, `buildStmtsDependencyGraph` y `enumerateSemanticTopSortsBounded`. También conviene mencionar los módulos agregados en `base` y `ghc-internal`.